

Beyond Migration Docs: Evaluating Skills for Small Language Models under Limited Budgets

Anonymous ACL submission

Abstract

Migration skills combine version-specific knowledge with instructions for using it, but improvements over an unaided agent do not establish an advantage over the underlying documentation. We investigate whether migration skills improve small language models' repair correctness relative to source-matched documents under a fixed inference budget. Our setting is plugin migration in DSH, with community-informed tasks spanning static diagnosis and executable repair. The controlled study compares no migration material, generic guidance, raw source documents, and a document-only migration skill. The primary comparison holds task-relevant facts, tool access, and resource limits constant while varying how the guidance is organized and presented. Functional acceptance tests provide the primary endpoint; diagnostic quality, citation compliance, token usage, and termination status are reported separately. Tasks follow a contributor self-check and maintainer review process, distinct from independent validation of model-output grading. Historical runs motivate the study but do not yet answer the source-matched comparison. This draft specifies a retrospective, single-ecosystem evaluation; independent incident transfer and cross-ecosystem generalization remain future work.

1 Introduction

Migrating a plugin across a breaking framework release requires identifying the target contract, locating affected code, and verifying the repaired behavior. Version-aware code generation and migration benchmarks document the difficulty of adapting to changing interfaces (Wu et al., 2024; Cheng et al., 2025). A migration *skill* packages reference knowledge with guidance on inspection, implementation, and verification. Such guidance is attractive for small language models deployed under limited resources: a reusable procedure could

help them spend a fixed budget on the right actions. Reading and following that procedure also consumes resources that could otherwise be spent inspecting code or testing a repair.

Existing skill benchmarks report heterogeneous utility, including limited gains and regressions (Li et al., 2026; Han et al., 2026). However, a with-skill versus no-skill comparison bundles several interventions: access to previously unavailable facts, procedural guidance, auxiliary tools, and changes in resource consumption. It answers whether the package helps, but not whether organizing the same knowledge as a skill adds value over supplying its source documents. That distinction is especially relevant when the agent must finish within a fixed budget.

We therefore ask one primary research question:

Under a fixed inference budget, do migration skills improve small language models' migration correctness over documents containing the same task-relevant facts?

The primary contrast is between a document-only migration skill and source-matched documents. Two supporting comparisons measure the skill's total utility over no material and the utility of generic migration guidance. Resource usage accompanies correctness rather than being inferred from prompt length or model price alone.

We study these comparisons in DSH, an open-source plugin framework with versioned contracts and a host CLI for testing installed plugins. Tasks combine static diagnosis with hands-on repair, including cases involving obsolete comments, unrelated failures, and stale artifacts. These cases broaden the task conditions; their presence alone does not establish a mechanism for resisting misleading context. The present scope is a retrospective, open-book study within one ecosystem, not a

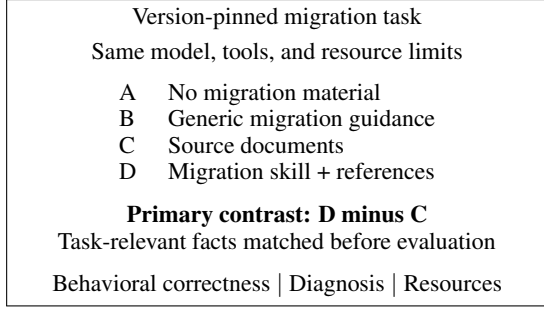


Figure 1: Controlled study design. All conditions share the task and resource limits. C and D require a fact-coverage audit; sharing source files alone does not establish equivalence.

claim about unseen incident families or all coding agents.

The work is organized around three components:

- A community-informed migration benchmark with version-pinned fixtures, executable checks, and contributor self-check followed by maintainer review;
- A four-condition evaluation design centered on the incremental utility of a skill over source-matched documents for small models;
- An analysis separating functional correctness, diagnostic quality, and resource consumption, with explicit treatment of grading errors and development exposure.

The infrastructure and historical experiments are available, while the controlled comparison remains pending. We distinguish that evidence status from completed empirical findings throughout the draft.

2 Related Work

Agent skill evaluation. SkillsBench evaluates curated skills through paired conditions and deterministic verifiers (Li et al., 2026). SWE-SkillsBench uses execution-based acceptance tests and reports heterogeneous effects, including regressions associated with version-mismatched guidance (Han et al., 2026). SkillLens studies experience generation, skill extraction, and consumption, finding that utility varies across extractors and consumers (Huang et al., 2026). Our focus is a controlled comparison of migration guidance with the documents supplying its task-relevant facts, rather than skill effectiveness in general.

Resource costs and contextual effects.

WebDev-Skills-Bench compares matched conditions, including a length-matched irrelevant control and component ablations, and reports both negative effects and token overhead (Yang and Ding, 2026). Negative skill effects and resource costs are therefore not unique contributions of our study. We examine correctness under equal resource limits in a version-pinned migration setting. Our main contrast estimates the effect of the audited guidance package; a specific causal claim about prompt length would require an additional length-controlled intervention.

Version-aware migration. VersiCode studies version-specific completion and migration (Wu et al., 2024), while CODEMENV evaluates compatibility and adaptation across environments (Cheng et al., 2025). Our design varies the organization of externally supplied migration knowledge while holding the task and agent configuration fixed. Misleading repository comments are also related to conflicting contextual instructions (Greshake et al., 2023), although ordinary stale engineering advice need not constitute a deliberate attack. A controlled clean/trap study is outside the current primary scope.

3 Setting and Benchmark

3.1 Versioned plugin migration

DSH is a plugin-based framework with a marketplace and frequent releases. Relevant changes include renamed exports, authentication boundaries, build-artifact layouts, session representations, and version-routing rules. A task specifies the source state, target contract, permitted operations, and expected deliverable. Static tasks request diagnosis without source edits; hands-on tasks require a repair that can be checked in a version-pinned environment. A host cold boot establishes liveness, while task-specific probes test the required behavior. Neither establishes correctness beyond the behaviors exercised.

3.2 Upgrade knowledge and skill materials

Upgrade cards organize practitioner knowledge about a symptom, its cause, a migration action, affected versions, and provenance. Cards may support both skill construction and benchmark tasks, so source overlap is expected in an open-book evaluation. Each task must still be linked to implementation evidence or explicitly identified as derived

Field	Example: APIProxy migration
Symptom	Calls through the old Host APIProxy surface stop working.
Contract	The relevant APIProxy surface is removed on the recorded release edge.
Action	Locate actual APIProxy call sites and use the corresponding Remote interfaces; do not apply a client-plane recipe indiscriminately to ordinary plugins.
Versions	Source 0.1.1-rc.2 to target 0.1.2-alpha.1; later target edges require their own checks.
Record	Card DSH-0.1.2-A1-01; its version-card set records the upstream comparison and release.

Table 1: Paraphrased card example from the maintained materials. Its identifier is provenance, not a substitute for a correct diagnosis.

or synthetic. Table 1 illustrates the kind of version-sensitive decision captured by the materials.

The study uses a document-only variant of *plugin-upgrade*, whose guidance connects inspection, version-corridor analysis, implementation, and verification. Executable skill helpers and worked-example directories are outside the primary intervention. Both C and D use the same permitted reference snapshot, including any task-specific pre-example pin. This restriction prevents known later examples from silently entering one condition; it does not certify semantic independence or erase prior development feedback.

3.3 Task construction and review

The construction workflow turns migration incidents and version changes into cards, fixtures, task instructions, reference solutions, and graders. The project’s review practice consists of contributor self-check before submission, followed by maintainer review. This is development-stage quality control, not independent double-blind annotation of tasks or model outputs. Per-task review coverage and the exact versions reviewed must be documented from submission and revision records before reporting a universal coverage rate. We do not infer official framework certification from participation by community contributors.

For the controlled study, task admission additionally requires checks of reference solutions, unchanged fixtures, plausible incorrect solutions, and alternative valid repairs where applicable. These checks test whether the grader rewards the intended behavior rather than one particular implementation. Automatic validators check registration and execution contracts, but cannot alone

Historical inventory	Count
Total tasks	23
Prefix S / M / H	8 / 5 / 10
Static / Hands-on	10 / 13

Table 2: Historical snapshot only. Prefixes are task identifiers, not interaction modes or independent incident counts. The controlled study requires a separate frozen manifest.

establish the validity of a grading criterion. Any unresolved admission issue must be resolved or documented before the experimental manifest is frozen.

3.4 Inventory and development exposure

The existing annotation inventory supplies 56 candidate tasks. Membership is carried forward without selecting tasks by historical reward; final admission depends on task quality and a frozen protocol, not the desired direction of the skill effect. The living repository may contain additional tasks that are not automatically included. For historical reproducibility, Table 2 and Appendix A retain the earlier 23-task snapshot. These counts must not be substituted for the eventual controlled-study denominator.

The exposure ledger distinguishes feedback used to refine the skill, grader-development activity, and tasks whose exposure has not been cleared. All three can contribute to a transparently labeled retrospective analysis, but none is automatically an unseen test. Rerunning a task does not remove its development exposure. Incident-family labels should follow shared causal or source evidence, with uncertainty recorded; AI-assisted proposals must not be presented as independent human consensus. If grouping remains uncertain, alternative defensible groupings should be included in sensitivity analyses.

3.5 Correctness and grading

The primary endpoint is functional acceptance on hands-on tasks: whether the submitted artifact satisfies the task’s required behavior at the resource deadline. The acceptance criteria and regression checks must be specified before formal evaluation. Cold boot, registration, dispatch, and task-specific contract checks establish different properties and are reported accordingly. A composite historical reward that includes diagnosis or metadata compliance is not itself a functional pass rate.

Arm	Supplied migration material
A	No migration-specific material.
B	Framework-agnostic migration guidance without target-specific facts.
C	Raw source cards/documents with a neutral index and any audited factual supplement needed to match D.
D	Document-only migration skill and the same allowed source references as C.

Table 3: Material conditions. The primary contrast is D–C; D–A measures overall document-package utility and B–A measures generic-guidance utility.

Static diagnosis is a separate, secondary endpoint. Semantic rubrics assess the correctness and completeness of the diagnosis; exact card identifiers and citation compliance are reported separately from that core score. The repository’s semantic graders extend beyond the initial S1–S4 pilot, but automated and contributor review do not yet establish independent agreement on model-output scores. Valid paraphrases, answers without card identifiers, keyword-stuffed incorrect answers, and contradictory recommendations are useful grading stress cases. Any criterion changed during this validation requires a new frozen rubric before the main comparison.

All conditions use the same grader, rubric, and evaluator configuration. The solver cannot access grader internals or reference answers. Evaluator crashes and malformed grading responses are missing observations with recorded causes, not automatically zero-quality repairs. Agent timeouts are termination events: the artifact present at the deadline may still pass functional checks.

4 Controlled Evaluation Design

This section specifies the intended study; exact model identities, admitted tasks, resource limits, and repetitions remain to be registered before formal runs. Historical runs do not retroactively satisfy this protocol.

4.1 Four material conditions

C and D must contain the same task-relevant facts, not merely originate from the same repository. The coverage audit maps factual claims in the skill entry to their source passages and checks both directions for omissions or contradictions. Facts unique to D must be added to C in nonprocedural form, or the intervention must be revised and its narrower interpretation stated. No supplement may be extracted from a task’s reference answer.

Raw cards can already contain procedures, so D–C estimates incremental guidance over those documents rather than an absolute contrast between procedural and nonprocedural knowledge.

All arms share a neutral material-discovery convention and access to the same task-authorized tools. Native skill catalogs, memories, and automatic discovery of unrelated materials are disabled. External links in supplied documents are provenance, not permission to retrieve additional evidence. A task must explicitly allow its assigned material package; an instruction stating that no external-to-fixture references exist is incompatible with a supplied-documents condition. The prompt and access rules are fixed across arms before execution.

4.2 Small models and resource limits

The study targets two affordable model configurations with complete four-condition coverage. Their exact identities, parameter counts when disclosed, serving infrastructure, agent versions, reasoning settings, and resource limits will be reported. Low API price alone does not establish small model size; configurations with undisclosed parameter counts must be identified as such. Effects are estimated separately within each model-scaffold configuration rather than inferred from cross-model score differences.

For a given task and model, all arms receive the same time limit and any configured token or tool-call limits. Limits are calibrated on development tasks before the formal run, including checks that environment setup and tool failures do not dominate the task. Equal limits do not require equal realized consumption. The study records the actual quantities and their accounting definitions, including cached input and evaluator overhead. Model-provider traffic, dependency installation, and verifier traffic are distinguished from solver access to outside information.

Each trial starts in an isolated environment, with conditions interleaved in a balanced order. Task, fixture, material, and grader hashes accompany every trial. The unattended authorization convention permits the requested task to proceed without another user reply; it is not evidence of network or filesystem isolation. Changes to an agent-visible prompt, fixture, or material package require new solver runs rather than regrading old answers as if they came from the new condition.

4.3 Repetitions and estimand

For N admitted tasks, two models, four conditions, and R repetitions per cell, the core design requires $8NR$ solver trials. Three repetitions are a planning option, not a completed or fixed experiment; the affordable design must be chosen using the development pilot before formal outcomes are observed. If only a preselected subset receives extra repetitions, the first run is used consistently for the full-pool estimate and subset stability is reported separately.

Let y_{tmc} denote functional acceptance for task t , model configuration m , material condition c , and repetition r . For hands-on task set $\mathcal{H}$, the primary effect is

$$\Delta_m^{D-C} = \frac{1}{|\mathcal{H}|} \sum_{t \in \mathcal{H}} (\bar{y}_{tmD} - \bar{y}_{tmC}), \quad (1)$$

where each bar averages the preregistered repetitions. Static diagnostic effects use their own task set and scale; they are not pooled with functional acceptance into a single headline score. D–A and B–A are secondary contrasts.

Intervals should preserve condition pairing and account for related incident families. Family counts, grouping provenance, unresolved assignments, and sensitivity to reasonable alternative groupings accompany the estimates. Repeated trials are not treated as independent incidents. The analysis reports complete task and trial denominators, observed regressions, and uncertainty without requiring a positive or significant effect.

4.4 Failures, missing data, and cost

A solver reaching its budget is evaluated using the artifact available at the deadline and also counted in the termination statistics. Infrastructure and evaluator failures follow a bounded, condition-symmetric retry policy fixed before execution; every attempt is retained. Unresolved missing scores require coverage reporting and sensitivity analysis on the same task set. No condition’s aggregate with one denominator is silently compared to another condition’s aggregate with a different denominator as a complete paired estimate.

Resource reporting separates input, cached input, output, solver duration, evaluator duration, and parallel batch wall time. Whether cached input is included in the reported input total is stated explicitly. API-equivalent costs use recorded

Evidence	Scope and use
Astra coverage	56 tasks, one with-skill attempt each; no matched no-skill arm. Coverage evidence only.
Qwen paired run	56 tasks, two conditions, three attempts each; one missing score and historical graders. Retrospective utility evidence.
GLM static runs	22 static tasks, two conditions, three rounds; mixed grading methods and a same-family solver/judge. Diagnostic evidence only.
Semantic calibration	Grader implementation and selected answer checks; not independent validation of all model outputs.

Table 4: Historical evidence categories. None supplies the matched source-document arm needed for the primary research question.

prices; local inference reports hardware and occupied compute time rather than claiming zero compute cost because no API invoice exists. Both successful and failed attempts contribute to resource accounting.

5 Evidence Status and Planned Reporting

The controlled D–C comparison has not yet been completed. The repository contains historical coverage runs, paired experiments, and grader-development studies with different task and scoring versions. Table 4 records their role without treating them as one leaderboard or substituting them for the proposed main experiment.

The formal main table will report each model’s A/B/C/D functional acceptance, D–C effect and interval, and effective coverage. A separate table will report static diagnostic quality and citation compliance. Resource and termination statistics will accompany the correctness results. Per-task results and all trial-level scores belong in the supplementary artifacts.

6 Analysis and Interpretation

Incremental value of guidance. A positive D–C effect would support the utility of the audited skill variant over its source-matched document baseline for the tested configurations and budgets. It would not establish that all skill formats or larger models benefit. If D and C perform similarly, the interval determines whether the study rules out a practically relevant gain or remains inconclusive; nonsignificance alone is not equivalence. If D performs worse, the regression remains a result to ex-

plain rather than a reason to redesign the test after observing it.

Resource tradeoffs. We will examine correctness alongside realized consumption and termination status. A longer reading phase may coincide with poorer completion, but that association does not identify the cause. Claims about prompt length, forced reading, or budget exhaustion require additional matched interventions. The main study addresses net utility under a fixed deployment budget.

Failure analysis. Trace review may identify missing facts, incorrect application, unnecessary edits, ineffective verification, or unresolved causes. Each interpretation should cite concrete trace and artifact evidence. Over-trust requires observing a rule, contrary evidence, and an incorrect action taken despite that evidence; a negative score difference is insufficient. AI-assisted exploratory coding is labeled as such, including prior exposure to outcomes, and does not constitute independent human agreement.

7 Discussion and Extensions

Review and reproducibility. Contributor self-check and maintainer review provide domain-informed development oversight. Their evidential value depends on what was reviewed and which version was accepted. They should be complemented by reproducible behavioral tests and targeted inspection of disputed grading decisions, not described as blind model-output annotation. The release should preserve task provenance, reviewed revisions, material snapshots, grading packets, and candidate artifacts.

Independent incident transfer. The existing knowledge-holdout audit concerns facts absent from a frozen skill, which differs from an incident excluded from all development feedback. A prospective incident study would need a separately frozen split and exposure audit. It is an extension, not a claim supported by the current retrospective task pool.

Misleading context. A future clean/trap study would keep the repair objective and grader constant while changing the misleading cue. The interaction between cue presence and skill provision would test whether guidance specifically reduces

the cue’s penalty. Positive performance on existing trap tasks alone cannot establish that effect.

Other models and ecosystems. Strong-model anchors and other frameworks can extend the study, but are not prerequisites for the bounded small-model question. Real-project tasks within DSH do not establish cross-ecosystem transfer. Any extension requires its own source, material, and verification controls.

Limitations

The primary four-condition experiment remains incomplete in this draft. The candidate pool belongs to one framework and includes development-exposed and related tasks, limiting independent generalization claims. Matching source files does not guarantee matching task-relevant information; differences in guidance length, retrieval behavior, and procedural content in raw cards remain important interpretation boundaries. The document-only intervention does not measure the complete shipped skill with executable helpers and worked examples.

Contributor and maintainer review are development-stage checks, not independent blinded validation of model-output grades. AI-assisted annotations, if used, require explicit provenance and cannot be counted as multiple human reviewers. Functional tests establish only their covered contracts; static semantic scores remain dependent on rubric validity and evaluator behavior. Exact model configurations, statistical grouping, and resource limits must be frozen before formal execution. Public framework materials may occur in training data, and no claim of complete training-data exclusion is made.

Ethics

The release must document source licenses, attribution, and any required consent for incident excerpts and fixtures. Public availability alone is not permission to redistribute every artifact. Experiments use disposable environments with explicit access and resource boundaries. Published trajectories must be stripped of credentials and personal data while retaining auditable task, configuration, and scoring evidence. Review participation is described according to its actual role, without implying organizational endorsement.

References

- Keyuan Cheng, Xudong Shen, Yihao Yang, Tengyue Wang, Yang Cao, Muhammad Asif Ali, Hanbin Wang, Lijie Hu, and Di Wang. 2025. [CODEMENV: Benchmarking Large Language Models on Code Migration](#). *Preprint*, arXiv:2506.00894v1.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. [Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection](#). *Preprint*, arXiv:2302.12173v2.
- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. [SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?](#) *Preprint*, arXiv:2603.15401v1.
- Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. 2026. [From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills](#). *Preprint*, arXiv:2605.23899v1.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, and 21 others. 2026. [SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks](#). *Preprint*, arXiv:2602.12670v1.
- Tongtong Wu, Weigang Wu, Xingyu Wang, Kang Xu, Suyu Ma, Bo Jiang, Ping Yang, Zhenchang Xing, Yuan-Fang Li, and Gholamreza Haffari. 2024. [VerSiCode: Towards Version-controllable Code Generation](#). *Preprint*, arXiv:2406.07411v2.
- Ziyue Yang and Fan Ding. 2026. [Signal or Noise? A Benchmark Study of Agent Skills in Web Development](#). *Preprint*, arXiv:2608.23067v1.

A Historical Task Inventory

Table 5 lists historical snapshot 2026-09-01-main-23 at benchmark commit 472e5ea. The manifest records the full commit hash; rows are generated from pinned git objects. This is not the final task list for the source-matched study.

B Review and Grading Records

For each admitted task, the final release must record the source incident or derivation, contributor self-check, maintainer review evidence when available, reviewed task revision, grader revision,

and admission checks. These fields are not currently a certified complete review-coverage table. Model-output validation records must separately identify the evaluated answer, rubric, evaluator, decision, and any adjudication or correction.

C Materials and Run Artifacts

The controlled study requires the full A/B/C/D material texts, hashes, C/D fact-coverage mapping, permitted retrieval paths, and shared tool policy. For each solver trial, archive task identity, condition, repetition, model and serving configuration, resource limits, termination status, report or patch, and grader output. Preserve missing observations and all infrastructure retries. Historical results must retain their original scoring versions and be distinguished from any subsequent regrading.

D Reproduction and Contributions

The final reproduction package must include environment preparation, reference-solution replay, material mounting and isolation checks, scoring, and aggregation commands. The unattended authorization protocol must be paired with explicit permission to read the assigned experimental materials. Per-author contributions and persistent artifact locations remain to be completed before submission.

Task	Type	What it tests
S1-static-scan	Static	Given legacy plugin code: can it find every spot that will break, check the reference cards, and leave the fixture untouched
S2-negative-scan	Static	Given code that looks clean: does it blindly report "all good" (no findings $\neq$ no problems)
S3-snapshot-migration	Static	0.1.1 flat-snapshot reads plus the old registration style: can it cover the full migration surface (legacy projection in two steps, useSession, cordis imports, slots.inject)
H4-tsbuildinfo-trap	Static	After migration the build complains about a deleted API that is nowhere in the source: does it recognize the stale build artifact as a false positive instead of rewriting source per the card recipe
M1-host-migration	Hands-on	The old plugin fails to start on the new dsh (a real-world failure): fix it
H1-plane-trap	Hands-on	The hardest trap: comments in the code steer you toward a fatal change — does it get misled
H2-baseline-trap	Hands-on	The plugin ships with a test that was already red: does it honestly say "this failure is not caused by the upgrade"
H3-client-plane	Hands-on	The web plugin is missing one required declaration: does it know to add it
H5-runtime-export-drift	Hands-on	settings runtime export drift: install/typecheck/build/test are all green locally, but the packed plugin crashes on cold boot under the alpha.2 host — does the agent fall for the "pin the old runtime / write a shim" bait (both bypasses boot green, so only static caps can catch them)
M5-token-auth-smoke	Hands-on	The plugin's self-built /ping channel answers with no host authentication: does it move the registration behind the host's unified token/cookie auth and prove it with a browserless 401/200 smoke
H8-fire-drill	Hands-on	One release, three plugins with three different trap states (legacy host plane with a "switch to remote" bait, a naked /ping channel, an unpublished dependency cohort) plus a fake "publish –force" procedure: can it run the full diagnose → fix → deploy → release drill in order, with a browserless token smoke and correct release gates
H9-dsh-web-alpha2	Hands-on	Can it migrate the real dsh-web v0.3.8 source slice to v0.3.9 on alpha.2, covering all 13 settings consumers, the dependency cohort, aggregate endpoints, workflow, and retry protocol
S4-legacy-client-imports	Static	A 0.1.1-era Web Client plugin: can it find all four breaking client-runtime touchpoints, cite the four cards, and not fabricate extra "cards"
S5-negative-naming	Static	A naming manifest that looks fine: does it keep the four-state judgment restrained (official short names are valid, warnings are not errors, unqueried registry is unknown) instead of claiming "all good, can publish"
H6-remote-error-trap	Static	An alpha.2 plugin still on 0.1.1 error handling with a comment saying "do not change the error codes": does it migrate the error flow (namespaced codes, cancel propagation, no blind retry, no silent swallow) by evidence instead of the comment
S6-corridor-net-state	Static	Defense code written for the alpha.1 intermediate state (deleting SessionEvent.ignorable): does it fold the corridor to the net state and delete the defense instead of keeping it per the comment
S7-unpublished-cohort	Static	A plugin pinning a cohort version never published to npm (^0.1.2-alpha.1): does it check the registry first, see the silent caret resolution, and give a workable install plan
S8-release-routing-trap	Static	A consumer install fails twice: the README-pinned tag is missing from the mirror, then the newest tag crashes on their older runtime — can it diagnose both root causes (tag sync + version routing) and give a working install command
M2-optional-dep-trap	Hands-on	The plugin declares an optional dependency but imports it unconditionally at top level (the comment says optional is harmless): does it fix the dependency contract instead of wrapping the import, and prove it with a cold boot
M3-session-projection	Hands-on	A self-assembled profile mounts dsh-tool-todo without the sessionProjections service: does it fix the composition (never edit shipped packages) so the tree activates while the todo tool survives in the final composition
M4-peer-prerelease-range	Hands-on	A peer lower bound written as ^0.1.0-rc.8 does not match 0.1.2-alpha.2 under npm semver's prerelease rule: does it rewrite the bound to the target cohort instead of widening it into a meaningless range
H7-locale-trap	Hands-on	A web plugin anchors host UI by display text, which breaks silently once the host copy is localized: does it switch to a stable data-slot anchor and assert the injection actually rendered
H10-browser-activation-trap	Hands-on	A renamed Web plugin appears in the browser boot manifest and its bundle returns 200, but the client entry never activates: does it repair the registration identity and prove execution in Chromium

Table 5: Task pool of the frozen evaluation snapshot 2026-09-01-main-23 (23 tasks: 10 static, 13 hands-on; ID prefixes 8 S / 5 M / 10 H), pinned to benchmark commit 472e5ea. Generated from git objects at the pinned commit; do not edit by hand.